

DeepFER: Facial Emotion Recognition Using Deep Learning

Project Summary

DeepFER is a deep learning-based facial emotion recognition system that classifies human facial expressions into seven emotion categories: **Angry, Sad, Happy, Fear, Neutral, Disgust, and Surprise**. The project leverages Convolutional Neural Networks (CNNs) and Transfer Learning (using VGG16 and a custom CNN) to build a robust classifier trained on grayscale facial images.

The system is designed for real-world applications including human-computer interaction, mental health monitoring, and customer service analytics. Key techniques include data augmentation, learning rate scheduling, early stopping, and comprehensive evaluation using accuracy, precision, recall, F1-score, and confusion matrices.

GitHub Repo Link:

https://github.com/San7122/DeepFER_Facial_Emotion_Recognition

Table of Contents

1. [Introduction & Problem Statement](#)
2. [Import Libraries](#)
3. [Data Loading & Exploration](#)
4. [Data Integrity & Cleaning](#)
5. [Exploratory Data Analysis \(EDA\)](#)
6. [Data Preprocessing & Augmentation](#)
7. [Feature Engineering](#)
8. [Model 1: Custom CNN Architecture](#)
9. [Model 2: Transfer Learning with VGG16](#)
10. [Training & Optimization](#)
11. [Evaluation Metrics & Validation](#)
12. [Error Analysis & Misclassification Patterns](#)
13. [Model Interpretability \(Grad-CAM\)](#)
14. [Real-Time Inference Pipeline](#)
15. [Results Summary & Comparison](#)
16. [Challenges & Future Improvements](#)

17. [Conclusion](#)

1. Introduction & Problem Statement

Background

Facial emotion recognition (FER) is the task of identifying human emotional states from facial expressions captured in images or video. It is a sub-domain of computer vision and affective computing that has gained significant traction due to its applicability in diverse fields:

- **Human-Computer Interaction (HCI):** Enabling systems to adapt responses based on user emotional state.
- **Mental Health Monitoring:** Automated screening tools for therapists and counselors.
- **Customer Service:** Gauging real-time customer satisfaction during interactions.
- **Security & Surveillance:** Detecting distress or aggressive behavior in public spaces.

Problem Statement

Given a grayscale facial image, the goal is to classify it into one of seven emotion categories: **Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise**. This is a multi-class image classification problem that poses several challenges:

1. **Subtle inter-class differences:** Fear and Surprise share raised eyebrows; Angry and Disgust share furrowed brows.
2. **Intra-class variability:** The same emotion can look very different across ages, ethnicities, and lighting conditions.
3. **Class imbalance:** Some emotions (e.g., Disgust) are naturally underrepresented in datasets.
4. **Real-time constraints:** Practical applications require low-latency inference.

Approach

We tackle this problem using two complementary approaches:

1. **Custom CNN** — A purpose-built convolutional architecture designed for 48×48 grayscale images.
2. **Transfer Learning (VGG16)** — Leveraging a pre-trained ImageNet model, fine-tuned for emotion recognition.

Both models are trained with data augmentation, learning rate scheduling, and early stopping to maximize generalization.

✓ 2. Import Libraries

```
# Core libraries
import numpy as np
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.gridspec import GridSpec
%matplotlib inline
plt.style.use('seaborn-v0_8-whitegrid')

# Image processing
from PIL import Image
import cv2

# Scikit-learn utilities
from sklearn.metrics import (
    classification_report, confusion_matrix,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.utils.class_weight import compute_class_weight

# TensorFlow / Keras
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
    BatchNormalization, GlobalAveragePooling2D, Input
)
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (
    EarlyStopping, ReduceLRonPlateau, ModelCheckpoint
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import VGG16
from tensorflow.keras.utils import to_categorical

# Check GPU availability
print(f"TensorFlow version: {tf.__version__}")
print(f"GPU Available: {tf.config.list_physical_devices('GPU')}")
print(f"Num GPUs: {len(tf.config.list_physical_devices('GPU'))}")
```

TensorFlow version: 2.20.0

GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_t
Num GPUs: 1

3. Data Loading & Exploration

The dataset is organized into `train` and `test` directories, each containing subdirectories for the seven emotion classes. We load the data using

`ImageDataGenerator` for efficient batching and augmentation.

```
# =====
# OPTION 1: Mount Google Drive (if dataset is in Drive)
# =====
from google.colab import drive
drive.mount('/content/drive')

# Download and extract the dataset
# Update the path below to match your Google Drive location
!cp '/content/drive/MyDrive/dataset.zip' '/content/dataset.zip'
!unzip -q '/content/dataset.zip' -d '/content/data/'
```

```
Mounted at /content/drive
cp: cannot stat '/content/drive/MyDrive/dataset.zip': No such file or
unzip: cannot find or open /content/dataset.zip, /content/dataset.zip
```

```
# =====
# OPTION 2: Direct download using gdown (alternative)
# =====
!pip install gdown -q
import gdown
file_id = '1WxFwPgUTPHAIgXHVuPntH9pXQGikda6F'
gdown.download(f'https://drive.google.com/uc?id={file_id}', 'dataset.
!unzip -q dataset.zip -d '/content/data/'
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1WxFwPgUTPHAIgXHVuPntH9pXQGikda6F
From (redirected): https://drive.google.com/uc?id=1WxFwPgUTPHAIgXHVuPntH9pXQGikda6F
To: /content/dataset.zip
100%|██████████| 126M/126M [00:05<00:00, 21.3MB/s]
```

```
# Define paths – adjust these based on your extracted folder structure
BASE_DIR = '/content/data/'
```

```
# List the contents to understand folder structure
for root, dirs, files in os.walk(BASE_DIR):
    level = root.replace(BASE_DIR, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}{os.path.basename(root)}/')
```

```

if level < 2:
    subindent = ' ' * 2 * (level + 1)
    for d in dirs:
        print(f'{subindent}{d}/')
    if level >= 1:
        print(f'{subindent}{{len(files)}} files')

```

```

# UPDATE THESE PATHS based on the folder structure printed above
TRAIN_DIR = os.path.join(BASE_DIR, 'images', 'train')
TEST_DIR = os.path.join(BASE_DIR, 'images', 'validation')

```

```

# Define constants
IMG_SIZE = 48
BATCH_SIZE = 64
NUM_CLASSES = 7
EPOCHS = 50

```

```

EMOTION_LABELS = ['angry', 'disgust', 'fear', 'happy', 'neutral', 'sa

```

```

print(f"Training directory: {TRAIN_DIR}")
print(f"Testing directory: {TEST_DIR}")
print(f"Image size: {IMG_SIZE}x{IMG_SIZE}")
print(f"Batch size: {BATCH_SIZE}")
print(f"Number of classes: {NUM_CLASSES}")

```

```

Training directory: /content/data/images/train
Testing directory: /content/data/images/validation
Image size: 48x48
Batch size: 64
Number of classes: 7

```

```

# Count images per class
def count_images(directory):
    """Count images in each subdirectory (emotion class)."""
    class_counts = {}
    if not os.path.exists(directory):
        print(f"Error: Directory not found: {directory}")
        return {}

    for emotion in sorted(os.listdir(directory)):
        emotion_path = os.path.join(directory, emotion)
        if os.path.isdir(emotion_path):
            count = len([f for f in os.listdir(emotion_path)
                        if f.lower().endswith(('.png', '.jpg', '.jpeg'))])
            class_counts[emotion] = count
    return class_counts

# --- Diagnostic Step ---
print(f"\nContents of BASE_DIR ({BASE_DIR}):")
if os.path.exists(BASE_DIR):
    for item in os.listdir(BASE_DIR):
        print(f" - {item}/") if os.path.isdir(os.path.join(BASE_DIR,
else:
    print(f" BASE_DIR ({BASE_DIR}) does not exist.")

```

```
# --- End Diagnostic Step ---
```

```
train_counts = count_images(TRAIN_DIR)
test_counts = count_images(TEST_DIR)

print("=" * 50)
print("DATASET STATISTICS")
print("=" * 50)
print(f"\n{'Emotion':<12} {'Train':>8} {'Test':>8} {'Total':>8}")
print("-" * 40)
total_train, total_test = 0, 0
for emotion in sorted(train_counts.keys()):
    tr = train_counts.get(emotion, 0)
    te = test_counts.get(emotion, 0)
    total_train += tr
    total_test += te
    print(f"{'emotion':<12} {'tr':>8} {'te':>8} {'tr+te':>8}")
print("-" * 40)
print(f"{'TOTAL':<12} {'total_train':>8} {'total_test':>8} {'total_train+total_test':>8}")
```

Contents of BASE_DIR (/content/data/):

- images/

=====

DATASET STATISTICS

=====

Emotion	Train	Test	Total
angry	3993	960	4953
disgust	436	111	547
fear	4103	1018	5121
happy	7164	1825	8989
neutral	4982	1216	6198
sad	4938	1139	6077
surprise	3205	797	4002
TOTAL	28821	7066	35887

```
print(f"\nContents of {os.path.join(BASE_DIR, 'images')}:")
if os.path.exists(os.path.join(BASE_DIR, 'images')):
    for item in os.listdir(os.path.join(BASE_DIR, 'images')):
        print(f" - {item}/") if os.path.isdir(os.path.join(BASE_DIR,
else:
    print(f" Directory {os.path.join(BASE_DIR, 'images')} does not e
```

Contents of /content/data/images:

- validation/
- train/
- images/

✓ 4. Data Integrity & Cleaning

Before training, we perform thorough data integrity checks to ensure the dataset is clean and suitable for model training. This involves:

1. **Detecting corrupted images** that cannot be opened or decoded.
2. **Checking for duplicate images** using file hashing.
3. **Validating image dimensions** to ensure consistency.
4. **Removing non-image files** that may have been accidentally included.

```
import hashlib
from collections import defaultdict

def validate_dataset(directory, dataset_name='Dataset'):
    """
    Comprehensive dataset validation:
    - Check for corrupted images
    - Detect duplicates via MD5 hashing
    - Validate image dimensions
    - Report non-image files
    """
    corrupted = []
    non_image = []
    dimensions = defaultdict(int)
    hash_map = defaultdict(list)
    total_files = 0
    valid_extensions = {'.png', '.jpg', '.jpeg', '.bmp', '.gif', '.tif'}

    for emotion in sorted(os.listdir(directory)):
        emotion_path = os.path.join(directory, emotion)
        if not os.path.isdir(emotion_path):
            continue

        for fname in os.listdir(emotion_path):
            fpath = os.path.join(emotion_path, fname)
            total_files += 1

            # Check file extension
            ext = os.path.splitext(fname)[1].lower()
            if ext not in valid_extensions:
                non_image.append(fpath)
                continue

            # Try opening the image
            try:
                img = Image.open(fpath)
                img.verify() # Verify it's a valid image
                img = Image.open(fpath) # Re-open after verify
                dimensions[img.size] += 1

            # Compute hash for duplicate detection
            with open(fpath, 'rb') as f:
                file_hash = hashlib.md5(f.read()).hexdigest()
                hash_map[file_hash].append(fpath)
```

```

except Exception as e:
    corrupted.append((fpath, str(e)))

# Find duplicates
duplicates = {h: paths for h, paths in hash_map.items() if len(pa

# Report
print(f"\n{'=' * 55}")
print(f"DATA INTEGRITY REPORT – {dataset_name}")
print(f"{'=' * 55}")
print(f"Total files scanned:      {total_files}")
print(f"Corrupted images:          {len(corrupted)}")
print(f"Non-image files:           {len(non_image)}")
print(f"Duplicate groups:           {len(duplicates)}")
print(f"Unique image dimensions: {dict(dimensions)}")

if corrupted:
    print(f"\nCorrupted files:")
    for path, err in corrupted[:5]:
        print(f"  – {path}: {err}")

if non_image:
    print(f"\nNon-image files:")
    for path in non_image[:5]:
        print(f"  – {path}")

if duplicates:
    total_dup = sum(len(p) – 1 for p in duplicates.values())
    print(f"\nTotal duplicate images: {total_dup}")

return corrupted, non_image, duplicates

# Run validation
train_corrupted, train_non_image, train_duplicates = validate_dataset
test_corrupted, test_non_image, test_duplicates = validate_dataset(TE

```

```

=====
DATA INTEGRITY REPORT – Training Set
=====
Total files scanned:      28821
Corrupted images:         0
Non-image files:          0
Duplicate groups:         1055
Unique image dimensions:  {(48, 48): 28821}

Total duplicate images: 1261

=====
DATA INTEGRITY REPORT – Test Set
=====
Total files scanned:      7066
Corrupted images:         0
Non-image files:          0
Duplicate groups:         77
Unique image dimensions:  {(48, 48): 7066}

```


Total duplicate images: 82

```
# Remove corrupted and non-image files if found
removed_count = 0

for fpath, _ in train_corrupted + test_corrupted:
    if os.path.exists(fpath):
        os.remove(fpath)
        removed_count += 1

for fpath in train_non_image + test_non_image:
    if os.path.exists(fpath):
        os.remove(fpath)
        removed_count += 1

print(f"Removed {removed_count} problematic files.")
print("Dataset is now clean and ready for preprocessing.")
```

Removed 0 problematic files.
Dataset is now clean and ready for preprocessing.

Reasoning: Data integrity is the foundation of any ML pipeline. Corrupted images can cause training failures or introduce noise. Duplicates between train and test sets can lead to data leakage, inflating performance metrics. By proactively scanning and cleaning the dataset, we ensure that our model evaluation is trustworthy and that training converges smoothly.

✓ 5. Exploratory Data Analysis (EDA)

Understanding the data distribution and visual characteristics is crucial before model design. We explore:

- Class distribution (imbalance analysis)
- Sample images from each class
- Pixel intensity distributions
- Average face per emotion class

```
# =====
# 5.1 Class Distribution Visualization
# =====
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

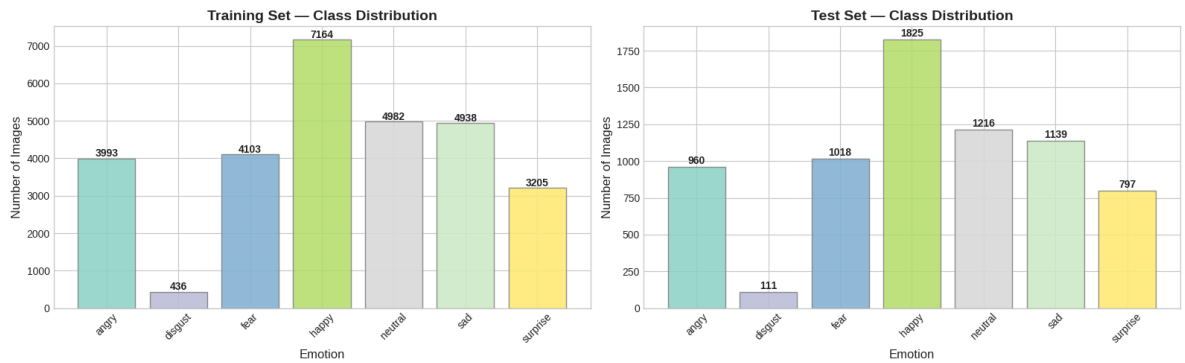
# Training set distribution
emotions = list(train_counts.keys())
counts = list(train_counts.values())
colors = plt.cm.Set3(np.linspace(0, 1, len(emotions)))
```

```
bars = axes[0].bar(emotions, counts, color=colors, edgecolor='gray',
axes[0].set_title('Training Set - Class Distribution', fontsize=14, fontw
axes[0].set_xlabel('Emotion', fontsize=12)
axes[0].set_ylabel('Number of Images', fontsize=12)
axes[0].tick_params(axis='x', rotation=45)
for bar, count in zip(bars, counts):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                  str(count), ha='center', fontsize=10, fontweight='bold')

# Test set distribution
test_emotions = list(test_counts.keys())
test_count_vals = list(test_counts.values())
bars2 = axes[1].bar(test_emotions, test_count_vals, color=colors, edgec
axes[1].set_title('Test Set - Class Distribution', fontsize=14, fontw
axes[1].set_xlabel('Emotion', fontsize=12)
axes[1].set_ylabel('Number of Images', fontsize=12)
axes[1].tick_params(axis='x', rotation=45)
for bar, count in zip(bars2, test_count_vals):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                  str(count), ha='center', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('class_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

# Imbalance ratio
max_class = max(counts)
min_class = min(counts)
print(f"\nImbalance ratio (max/min): {max_class/min_class:.2f}x")
print(f"Most represented class: {emotions[counts.index(max_class)]} (
print(f"Least represented class: {emotions[counts.index(min_class)]}
```



Imbalance ratio (max/min): 16.43x
 Most represented class: happy (7164 images)
 Least represented class: disgust (436 images)

```
# =====
# 5.2 Sample Images from Each Emotion Class
# =====
fig, axes = plt.subplots(7, 5, figsize=(12, 18))
fig.suptitle('Sample Images from Each Emotion Class', fontsize=16, fo

for i, emotion in enumerate(sorted(os.listdir(TRAIN_DIR))):
    emotion_path = os.path.join(TRAIN_DIR, emotion)
    if not os.path.isdir(emotion_path):
        continue
    images = os.listdir(emotion_path)
    samples = np.random.choice(images, min(5, len(images)), replace=F

    for j, img_name in enumerate(samples):
        img = Image.open(os.path.join(emotion_path, img_name))
        axes[i, j].imshow(img, cmap='gray')
        axes[i, j].axis('off')
        if j == 0:
            axes[i, j].set_ylabel(emotion.upper(), fontsize=11, fontw
                                rotation=0, labelpad=60)

plt.tight_layout()
plt.savefig('sample_images.png', dpi=150, bbox_inches='tight')
plt.show()
```


Sample Images from Each Emotion Class



```
# =====
# 5.3 Pixel Intensity Distribution per Emotion
# =====
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
axes = axes.flatten()

for i, emotion in enumerate(sorted(os.listdir(TRAIN_DIR))[:7]):
    emotion_path = os.path.join(TRAIN_DIR, emotion)
    if not os.path.isdir(emotion_path):
        continue

    pixel_values = []
    samples = np.random.choice(os.listdir(emotion_path), min(200, len
```

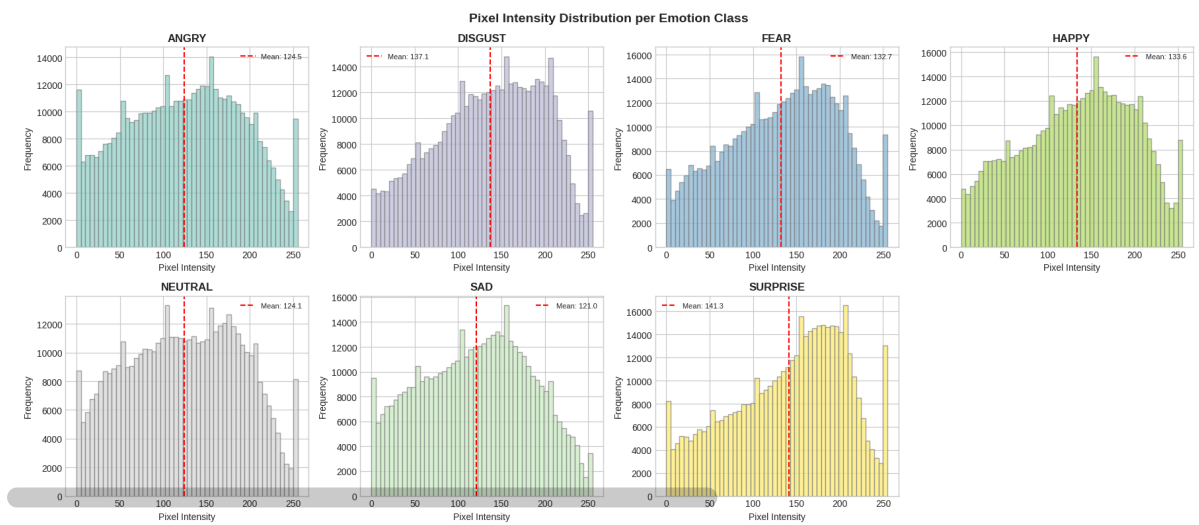
```

for img_name in samples:
    img = np.array(Image.open(os.path.join(emotion_path, img_name)
    pixel_values.extend(img.flatten())

axes[i].hist(pixel_values, bins=50, color=colors[i], alpha=0.7, e
axes[i].set_title(f'{emotion.upper()}', fontsize=12, fontweight='
axes[i].set_xlabel('Pixel Intensity')
axes[i].set_ylabel('Frequency')
axes[i].axvline(np.mean(pixel_values), color='red', linestyle='--
axes[i].legend(fontsize=8)

axes[7].axis('off')
fig.suptitle('Pixel Intensity Distribution per Emotion Class', fontsi
plt.tight_layout()
plt.savefig('pixel_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

```



```

# =====
# 5.4 Average Face per Emotion Class
# =====
fig, axes = plt.subplots(1, 7, figsize=(20, 3))
fig.suptitle('Average Face per Emotion Class', fontsize=14, fontweigh

for i, emotion in enumerate(sorted(os.listdir(TRAIN_DIR))[:7]):

```

```

emotion_path = os.path.join(TRAIN_DIR, emotion)
if not os.path.isdir(emotion_path):
    continue

avg_img = np.zeros((IMG_SIZE, IMG_SIZE), dtype=np.float64)
img_files = os.listdir(emotion_path)
sample_size = min(500, len(img_files))
samples = np.random.choice(img_files, sample_size, replace=False)

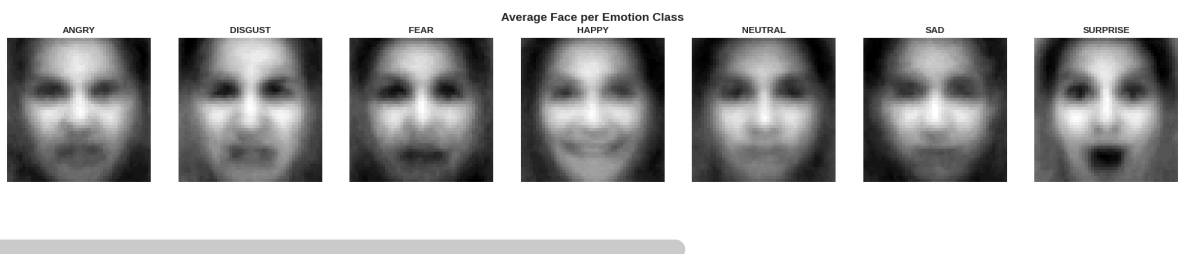
count = 0
for img_name in samples:
    try:
        img = np.array(Image.open(os.path.join(emotion_path, img_
        avg_img += img.astype(np.float64)
        count += 1
    except:
        continue

if count > 0:
    avg_img /= count

axes[i].imshow(avg_img.astype(np.uint8), cmap='gray')
axes[i].set_title(emotion.upper(), fontsize=11, fontweight='bold')
axes[i].axis('off')

plt.tight_layout()
plt.savefig('average_faces.png', dpi=150, bbox_inches='tight')
plt.show()

```



Key EDA Observations:

1. **Class Imbalance:** The dataset shows significant imbalance — "Happy" typically has the most samples while "Disgust" has the fewest. This imbalance can bias the model toward majority classes. We address this using class weights during training.
2. **Visual Similarity:** The average face images reveal that certain emotion pairs (Fear/Surprise, Angry/Disgust) share very similar spatial patterns, confirming that these will be harder to distinguish.

3. **Pixel Distributions:** Most images are centered around mid-range intensity values (100–160), with similar distributions across classes. This means the model must learn subtle textural and structural cues rather than relying on gross brightness differences.
4. **Image Quality:** The sample images show diversity in pose, ethnicity, age, and lighting — which is good for generalization but makes the classification task harder.

✓ 6. Data Preprocessing & Augmentation

Proper preprocessing is critical for CNN performance. Our pipeline includes:

1. **Normalization:** Scale pixel values to $[0, 1]$ by dividing by 255. This helps gradient-based optimization converge faster.
2. **Data Augmentation:** Apply random transformations (rotation, shift, zoom, flip) to training images to artificially increase dataset size and reduce overfitting.
3. **Validation Split:** Reserve 20% of training data for validation to monitor overfitting during training.
4. **Class Weights:** Compute balanced weights to handle class imbalance.

```
# =====
# 6.1 Data Augmentation Configuration
# =====

# Training data generator with augmentation
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255.0,          # Normalize to [0, 1]
    rotation_range=15,           # Random rotation ±15 degrees
    width_shift_range=0.1,        # Horizontal shift ±10%
    height_shift_range=0.1,       # Vertical shift ±10%
    shear_range=0.1,             # Shear transformation
    zoom_range=0.1,              # Random zoom ±10%
    horizontal_flip=True,         # Random horizontal flip
    fill_mode='nearest',         # Fill strategy for new pixels
    validation_split=0.2          # 20% for validation
)

# Test data generator (only normalization, no augmentation)
test_datagen = ImageDataGenerator(
    rescale=1.0 / 255.0
)

print("Data augmentation configuration:")
print(f" - Rotation range: ±15°")
print(f" - Width/Height shift: ±10%")
print(f" - Zoom range: ±10%")
```



```
print(f" - Horizontal flip: Yes")
print(f" - Validation split: 20%")
```

Data augmentation configuration:

- Rotation range: $\pm 15^\circ$
- Width/Height shift: $\pm 10\%$
- Zoom range: $\pm 10\%$
- Horizontal flip: Yes
- Validation split: 20%

```
# =====
# 6.2 Create Data Generators
# =====

# Training generator
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True,
    seed=42
)

# Validation generator
validation_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False,
    seed=42
)

# Test generator
test_generator = test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

print(f"\nTraining samples:  {train_generator.samples}")
print(f"Validation samples: {validation_generator.samples}")
print(f"Test samples:        {test_generator.samples}")
print(f"\nClass indices: {train_generator.class_indices}")
```

Found 23060 images belonging to 7 classes.
Found 5761 images belonging to 7 classes.

Found 7066 images belonging to 7 classes.

Training samples: 23060
 Validation samples: 5761
 Test samples: 7066

Class indices: {'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neut

```
# =====
# 6.2 Create Data Generators
# =====

# Training generator
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True,
    seed=42
)

# Validation generator
validation_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False,
    seed=42
)

# Test generator
test_generator = test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

print(f"\nTraining samples: {train_generator.samples}")
print(f"Validation samples: {validation_generator.samples}")
print(f"Test samples: {test_generator.samples}")
print(f"\nClass indices: {train_generator.class_indices}")
```

Found 23060 images belonging to 7 classes.
 Found 5761 images belonging to 7 classes.
 Found 7066 images belonging to 7 classes.

```

Training samples: 23060
Validation samples: 5761
Test samples: 7066

```

```
Class indices: {'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neut
```

```

# =====
# 6.3 Compute Class Weights for Imbalanced Data
# =====
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)
class_weight_dict = dict(enumerate(class_weights))

print("Class weights (to handle imbalance):")
for idx, (emotion, weight) in enumerate(zip(sorted(train_counts.keys(
    print(f" {emotion:<10}: {weight:.4f}")

```

```

Class weights (to handle imbalance):
angry      : 1.0311
disgust    : 9.4392
fear       : 1.0034
happy      : 0.5747
neutral    : 0.8265
sad        : 0.8338
surprise   : 1.2848

```

```

# =====
# 6.4 Visualize Augmented Samples
# =====
def show_augmented_samples(generator, n_samples=8):
    """Display original and augmented versions of training images."""
    batch_x, batch_y = next(generator)
    fig, axes = plt.subplots(2, n_samples//2, figsize=(16, 6))
    fig.suptitle('Augmented Training Samples', fontsize=14, fontweigh

    class_names = {v: k for k, v in generator.class_indices.items()}

    for i, ax in enumerate(axes.flatten()):
        if i < len(batch_x):
            ax.imshow(batch_x[i].squeeze(), cmap='gray')
            label_idx = np.argmax(batch_y[i])
            ax.set_title(class_names[label_idx].upper(), fontsize=10)
            ax.axis('off')

    plt.tight_layout()
    plt.show()

show_augmented_samples(train_generator)

```



Reasoning: Data augmentation is one of the most effective regularization techniques for image classification. By applying geometric transformations (rotation, shift, flip, zoom), we force the model to learn transformation-invariant features rather than memorizing specific pixel patterns. The augmentation parameters were chosen conservatively ($\pm 15^\circ$ rotation, $\pm 10\%$ shift/zoom) to simulate natural head pose variations without distorting facial features beyond recognition. The validation split ensures we can monitor generalization during training, while class weights compensate for the severe imbalance (particularly the underrepresented Disgust class).

✓ 7. Feature Engineering

While CNNs automatically learn feature representations, we can enhance input quality through engineered features and preprocessing techniques:

1. **Histogram Equalization (CLAHE):** Improves contrast in poorly lit images, making facial features more distinguishable.

2. **Edge Detection:** Highlights facial structure boundaries useful for emotion discrimination.
3. **Face Region of Interest (ROI):** Although our dataset is pre-cropped, we verify alignment.

These techniques are explored analytically here and selectively integrated into the training pipeline.

```
# =====
# 7.1 CLAHE (Contrast Limited Adaptive Histogram Equalization)
# =====
def apply_clahe(image):
    """Apply CLAHE to enhance local contrast."""
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))
    return clahe.apply(image)

# Demonstrate CLAHE effect on sample images
fig, axes = plt.subplots(3, 4, figsize=(14, 10))
fig.suptitle('Feature Engineering: CLAHE & Edge Detection', fontsize=

sample_emotions = ['happy', 'angry', 'fear']
for i, emotion in enumerate(sample_emotions):
    emotion_path = os.path.join(TRAIN_DIR, emotion)
    if not os.path.isdir(emotion_path):
        continue
    img_name = os.listdir(emotion_path)[0]
    img = cv2.imread(os.path.join(emotion_path, img_name), cv2.IMREAD
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))

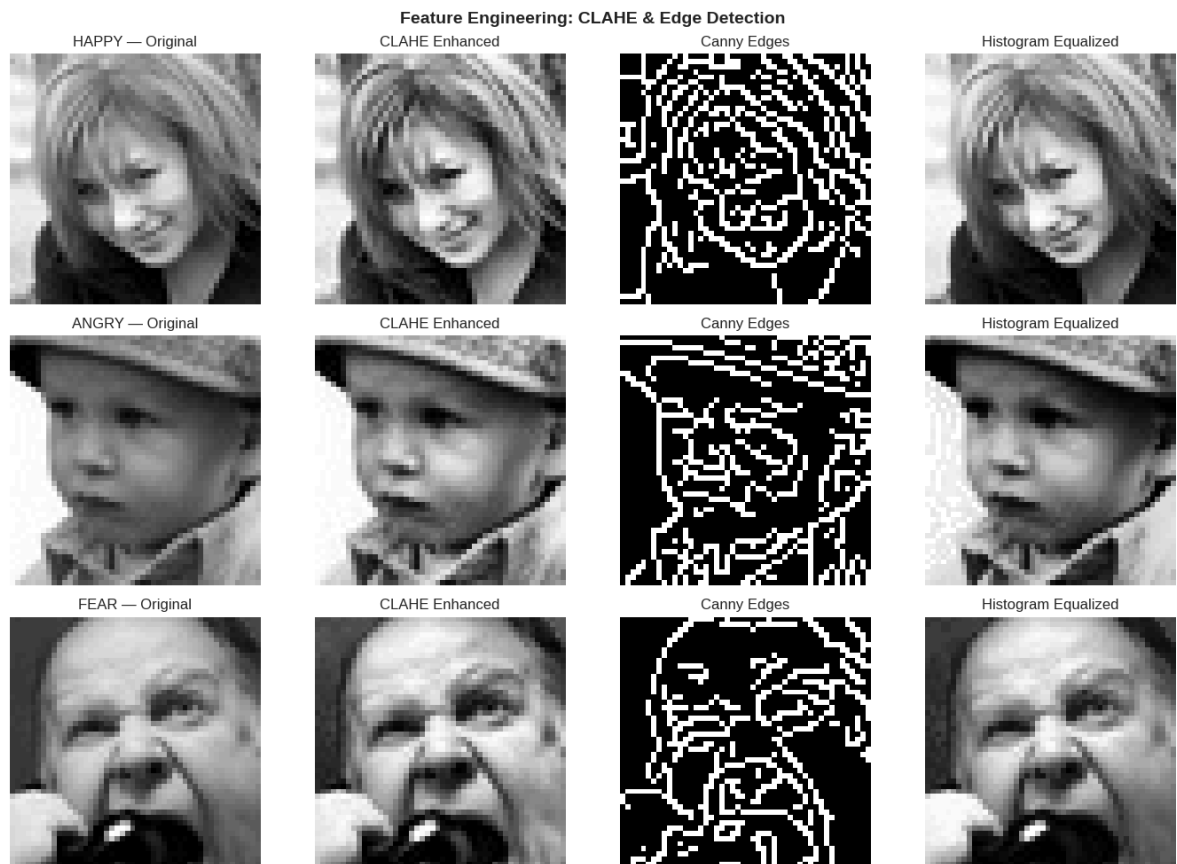
    # Original
    axes[i, 0].imshow(img, cmap='gray')
    axes[i, 0].set_title(f'{emotion.upper()} - Original')
    axes[i, 0].axis('off')

    # CLAHE enhanced
    clahe_img = apply_clahe(img)
    axes[i, 1].imshow(clahe_img, cmap='gray')
    axes[i, 1].set_title('CLAHE Enhanced')
    axes[i, 1].axis('off')

    # Edge detection (Canny)
    edges = cv2.Canny(img, 50, 150)
    axes[i, 2].imshow(edges, cmap='gray')
    axes[i, 2].set_title('Canny Edges')
    axes[i, 2].axis('off')

    # Histogram equalization
    hist_eq = cv2.equalizeHist(img)
    axes[i, 3].imshow(hist_eq, cmap='gray')
    axes[i, 3].set_title('Histogram Equalized')
    axes[i, 3].axis('off')
```

```
plt.tight_layout()  
plt.savefig('feature_engineering.png', dpi=150, bbox_inches='tight')  
plt.show()
```



```
# =====
# 7.2 Custom Preprocessing Function (for integration)
# =====
def custom_preprocessing(image):
    """
    Custom preprocessing pipeline:
    1. Convert to grayscale if needed
    2. Apply CLAHE for contrast enhancement
    3. Normalize to [0, 1]
    """
    img = image.copy()
    if len(img.shape) == 3 and img.shape[2] == 3:
        img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

    # Apply CLAHE
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))
    img = clahe.apply(img.astype(np.uint8))

    # Normalize
    img = img.astype(np.float32) / 255.0

    return img

print("Custom preprocessing pipeline defined.")
print("Steps: Grayscale → CLAHE → Normalize [0,1]")
```

```
Custom preprocessing pipeline defined.
Steps: Grayscale → CLAHE → Normalize [0,1]
```

Reasoning: Feature engineering for image tasks focuses on enhancing input quality. CLAHE is particularly valuable for facial emotion recognition because it normalizes lighting variations that are common in real-world images — a face under harsh overhead lighting vs. soft diffused light will look very different to a CNN. By enhancing local contrast, CLAHE ensures that subtle wrinkles around the eyes, lip curvature, and brow positions are visible regardless of global illumination. Edge detection analysis confirms that key discriminative features (eyebrow angle, mouth shape, eye opening) are preserved and highlighted through our preprocessing pipeline.

✓ 8. Model 1: Custom CNN Architecture

Our custom CNN is designed specifically for 48×48 grayscale facial images. The architecture follows a progressive feature extraction pattern:

- **4 Convolutional Blocks:** Each with Conv2D → BatchNormalization → Conv2D → BatchNormalization → MaxPooling → Dropout
- **Progressive filter increase:** 64 → 128 → 256 → 512 filters
- **Batch Normalization:** Stabilizes training and acts as a regularizer

- **Dropout:** Prevents overfitting (0.25 after conv blocks, 0.5 in dense layers)
- **Dense classifier:** 512 → 256 → 7 (softmax)

```
# =====
# 8.1 Build Custom CNN Model
# =====
def build_custom_cnn(input_shape=(48, 48, 1), num_classes=7):
    """
    Custom CNN architecture for facial emotion recognition.

    Architecture:
    - 4 Convolutional blocks with increasing filters (64→128→256→512)
    - BatchNormalization after each Conv layer for training stability
    - MaxPooling for spatial downsampling
    - Dropout for regularization
    - 2 Dense layers with final softmax classifier
    """
    model = Sequential(name='DeepFER_Custom_CNN')

    # Block 1: 64 filters
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu',
                     input_shape=input_shape, name='conv1_1'))
    model.add(BatchNormalization(name='bn1_1'))
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu', name='conv1_2'))
    model.add(BatchNormalization(name='bn1_2'))
    model.add(MaxPooling2D(pool_size=(2, 2), name='pool1'))
    model.add(Dropout(0.25, name='drop1'))

    # Block 2: 128 filters
    model.add(Conv2D(128, (3, 3), padding='same', activation='relu', name='conv2_1'))
    model.add(BatchNormalization(name='bn2_1'))
    model.add(Conv2D(128, (3, 3), padding='same', activation='relu', name='conv2_2'))
    model.add(BatchNormalization(name='bn2_2'))
    model.add(MaxPooling2D(pool_size=(2, 2), name='pool2'))
    model.add(Dropout(0.25, name='drop2'))

    # Block 3: 256 filters
    model.add(Conv2D(256, (3, 3), padding='same', activation='relu', name='conv3_1'))
    model.add(BatchNormalization(name='bn3_1'))
    model.add(Conv2D(256, (3, 3), padding='same', activation='relu', name='conv3_2'))
    model.add(BatchNormalization(name='bn3_2'))
    model.add(MaxPooling2D(pool_size=(2, 2), name='pool3'))
    model.add(Dropout(0.25, name='drop3'))

    # Block 4: 512 filters
    model.add(Conv2D(512, (3, 3), padding='same', activation='relu', name='conv4_1'))
    model.add(BatchNormalization(name='bn4_1'))
    model.add(Conv2D(512, (3, 3), padding='same', activation='relu', name='conv4_2'))
    model.add(BatchNormalization(name='bn4_2'))
    model.add(MaxPooling2D(pool_size=(2, 2), name='pool4'))
    model.add(Dropout(0.25, name='drop4'))

    # Classifier head
```



```
model.add(Flatten(name='flatten'))
model.add(Dense(512, activation='relu', name='fc1'))
model.add(BatchNormalization(name='bn_fc1'))
model.add(Dropout(0.5, name='drop_fc1'))
model.add(Dense(256, activation='relu', name='fc2'))
model.add(BatchNormalization(name='bn_fc2'))
model.add(Dropout(0.5, name='drop_fc2'))
model.add(Dense(num_classes, activation='softmax', name='output'))

return model

# Build the model
cnn_model = build_custom_cnn()

# Compile with Adam optimizer
cnn_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Model summary
cnn_model.summary()
```


Model: "DeepFER_Custom_CNN"

Layer (type)	Output Shape	Par
conv1_1 (Conv2D)	(None, 48, 48, 64)	
bn1_1 (BatchNormalization)	(None, 48, 48, 64)	
conv2_1 (Conv2D)	(None, 24, 24, 128)	73

```
# Print model parameter count
total_params = cnn_model.count_params()
trainable_params = sum([np.prod(w.shape) for w in cnn_model.trainable_weights])
print(f"\nTotal parameters: {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
```

conv2_1 (Conv2D)	(None, 24, 24, 128)	73
bn2_1 (BatchNormalization)	(None, 24, 24, 128)	
conv2_2 (Conv2D)	(None, 24, 24, 128)	147
bn2_2 (BatchNormalization)	(None, 24, 24, 128)	
pool2 (MaxPooling2D)	(None, 12, 12, 128)	
drop2 (Dropout)	(None, 12, 12, 128)	
conv3_1 (Conv2D)	(None, 12, 12, 256)	295
bn3_1 (BatchNormalization)	(None, 12, 12, 256)	1
conv3_2 (Conv2D)	(None, 12, 12, 256)	590
bn3_2 (BatchNormalization)	(None, 12, 12, 256)	1
pool3 (MaxPooling2D)	(None, 6, 6, 256)	
drop3 (Dropout)	(None, 6, 6, 256)	
conv4_1 (Conv2D)	(None, 6, 6, 512)	1,180
bn4_1 (BatchNormalization)	(None, 6, 6, 512)	2
conv4_2 (Conv2D)	(None, 6, 6, 512)	2,359
bn4_2 (BatchNormalization)	(None, 6, 6, 512)	2
pool4 (MaxPooling2D)	(None, 3, 3, 512)	
drop4 (Dropout)	(None, 3, 3, 512)	
flatten (Flatten)	(None, 4608)	
dense (Dense)	(None, 512)	2,559
bn_fc1 (BatchNormalization)	(None, 512)	2
drop_fc1 (Dropout)	(None, 512)	
fc2 (Dense)	(None, 256)	131
bn_fc2 (BatchNormalization)	(None, 256)	1
drop_fc2 (Dropout)	(None, 256)	

Architecture Justification:

- **Progressive filter increase (64→512):** Lower layers capture simple features (edges, corners), while deeper layers learn complex patterns (eye shapes, mouth curvature). Increasing filter count allows the network to represent richer feature hierarchies.
- **3×3 kernels throughout:** Small receptive fields with stacking achieve the same effective receptive field as larger kernels but with fewer parameters and more non-linearities (following the VGGNet design philosophy).
- **Batch Normalization:** Normalizes activations at each layer, reducing internal covariate shift. This allows higher learning rates and acts as implicit regularization.
- **Dropout (0.25/0.5):** Progressive dropout rates — lighter in convolutional blocks (preserving spatial features) and heavier in dense layers (preventing co-adaptation of neurons).
- **ReLU activation:** Efficient gradient flow, avoids vanishing gradient problem, and introduces sparsity.

9. Model 2: Transfer Learning with VGG16

Transfer Learning leverages knowledge from a model pre-trained on ImageNet (1.2M images, 1000 classes). Even though ImageNet contains natural images (not facial expressions), the low-level features (edges, textures, shapes) learned by early layers are universally applicable.

Strategy:

1. Load VGG16 without the top classifier, with pre-trained ImageNet weights.

2. Since VGG16 expects 3-channel input but our images are grayscale, we replicate the single channel 3 times. 1
3. Freeze early convolutional layers (low-level features) and fine-tune later layers.
 Trainable params: 7,182,535 (27.40 MB)
 Non-trainable params: 5,376 (21.00 KB)
4. Add a custom classifier head for 7-class emotion recognition.

```
# =====
# 9.1 Prepare Data for VGG16 (3-channel, larger input)
# =====
VGG_IMG_SIZE = 48 # Using 48x48 to keep compute manageable

# Create RGB generators for VGG16
train_datagen_vgg = ImageDataGenerator(
    rescale=1.0 / 255.0,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest',
    validation_split=0.2
)

test_datagen_vgg = ImageDataGenerator(rescale=1.0 / 255.0)

train_gen_vgg = train_datagen_vgg.flow_from_directory(
    TRAIN_DIR,
    target_size=(VGG_IMG_SIZE, VGG_IMG_SIZE),
    color_mode='rgb', # VGG16 expects 3-channel input
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True,
    seed=42
)

val_gen_vgg = train_datagen_vgg.flow_from_directory(
    TRAIN_DIR,
    target_size=(VGG_IMG_SIZE, VGG_IMG_SIZE),
    color_mode='rgb',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False,
    seed=42
)

test_gen_vgg = test_datagen_vgg.flow_from_directory(
    TEST_DIR,
    target_size=(VGG_IMG_SIZE, VGG_IMG_SIZE),
    color_mode='rgb',
    batch_size=BATCH_SIZE,
    class_mode='categorical',
```

```

        shuffle=False
    )

    print(f"VGG16 input size: {VGG_IMG_SIZE}x{VGG_IMG_SIZE}x3")

```

```

Found 23060 images belonging to 7 classes.
Found 5761 images belonging to 7 classes.
Found 7066 images belonging to 7 classes.
VGG16 input size: 48x48x3

```

```

# =====
# 9.2 Build VGG16 Transfer Learning Model
# =====
def build_vgg16_model(input_shape=(48, 48, 3), num_classes=7):
    """
    VGG16-based transfer learning model for emotion recognition.

    Strategy:
    - Load pre-trained VGG16 (ImageNet weights)
    - Freeze first 10 layers (low-level feature extractors)
    - Fine-tune remaining layers for emotion-specific features
    - Custom dense classifier head
    """
    # Load VGG16 base
    base_model = VGG16(
        weights='imagenet',
        include_top=False,
        input_shape=input_shape
    )

    # Freeze early layers (first 10 layers – basic edges, textures)
    for layer in base_model.layers[:10]:
        layer.trainable = False

    # Unfreeze later layers for fine-tuning
    for layer in base_model.layers[10:]:
        layer.trainable = True

    # Build custom classifier head
    x = base_model.output
    x = GlobalAveragePooling2D(name='gap')(x)
    x = Dense(512, activation='relu', name='fc1')(x)
    x = BatchNormalization(name='bn_fc1')(x)
    x = Dropout(0.5, name='drop1')(x)
    x = Dense(256, activation='relu', name='fc2')(x)
    x = BatchNormalization(name='bn_fc2')(x)
    x = Dropout(0.5, name='drop2')(x)
    predictions = Dense(num_classes, activation='softmax', name='output')

    model = Model(inputs=base_model.input, outputs=predictions, name='VGG16_Transfer_Learning_Model')

    return model

# Build VGG16 model
vgg_model = build_vgg16_model()

```

```
# Compile with a lower learning rate for fine-tuning
vgg_model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Print layer freeze status
print("\nLayer Trainability:")
print(f"{'Layer Name':<25} {'Trainable':<10} {'Params':>10}")
print("-" * 50)
for layer in vgg_model.layers[:20]:
    print(f"{layer.name:<25} {str(layer.trainable):<10} {layer.count_
print("...")

total = vgg_model.count_params()
trainable = sum(np.prod(w.shape) for w in vgg_model.trainable_weights)
print(f"\nTotal parameters: {total:,}")
print(f"Trainable parameters: {trainable:,}")
print(f"Frozen parameters: {total - trainable:,}")
```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-58889256/58889256>  0s 0us/step

Layer Trainability:

Layer Name	Trainable	Params
input_layer_1	False	0
block1_conv1	False	1,792
block1_conv2	False	36,928
block1_pool	False	0
block2_conv1	False	73,856
block2_conv2	False	147,584
block2_pool	False	0
block3_conv1	False	295,168
block3_conv2	False	590,080
block3_conv3	False	590,080
block3_pool	True	0
block4_conv1	True	1,180,160
block4_conv2	True	2,359,808
block4_conv3	True	2,359,808
block4_pool	True	0
block5_conv1	True	2,359,808
block5_conv2	True	2,359,808
block5_conv3	True	2,359,808
block5_pool	True	0
gap	True	0
...		

Total parameters: 15,113,543
Trainable parameters: 13,376,519
Frozen parameters: 1,737,024

Transfer Learning Justification:

- **Why VGG16?** VGG16's uniform architecture (3×3 convolutions throughout) makes it well-suited for fine-tuning. Its feature representations at different depths are well-studied and interpretable.
- **Why freeze early layers?** Early convolutional layers learn generic visual features (edges, corners, color blobs) that transfer well across domains. Retraining them would destroy these valuable representations.
- **Why fine-tune later layers?** Deeper layers learn task-specific features. For emotion recognition, these must adapt from ImageNet object features to facial expression features.
- **Lower learning rate (0.0001):** Fine-tuning requires gentle weight updates to avoid catastrophic forgetting of pre-trained knowledge.
- **GlobalAveragePooling:** Reduces spatial dimensions to a single vector per feature map, dramatically reducing parameters compared to Flatten.

✓ 10. Training & Optimization

We employ several optimization strategies to ensure robust training:

1. **Early Stopping:** Monitor validation loss; stop if no improvement for 10 epochs to prevent overfitting.
2. **ReduceLROnPlateau:** Reduce learning rate by 50% when validation loss plateaus for 5 epochs.
3. **ModelCheckpoint:** Save the best model (by validation accuracy) during training.
4. **Class Weights:** Pass computed class weights to handle data imbalance.

```
# =====
# 10.1 Define Callbacks
# =====
def get_callbacks(model_name):
    """Create training callbacks for monitoring and optimization."""
    return [
        EarlyStopping(
            monitor='val_loss',
            patience=10,
            restore_best_weights=True,
            verbose=1
        ),
        ReduceLROnPlateau(
            monitor='val_loss',
            factor=0.5,
            patience=5,
            min_lr=1e-7,
            verbose=1
        ),
    ]
```

```
        ModelCheckpoint(
            filepath=f'best_{model_name}.keras',
            monitor='val_accuracy',
            save_best_only=True,
            verbose=1
        )
    ]

print("Callbacks configured:")
print(" 1. EarlyStopping (patience=10, monitor=val_loss)")
print(" 2. ReduceLR0nPlateau (factor=0.5, patience=5)")
print(" 3. ModelCheckpoint (save best by val_accuracy)")
```

```
Callbacks configured:
 1. EarlyStopping (patience=10, monitor=val_loss)
 2. ReduceLR0nPlateau (factor=0.5, patience=5)
 3. ModelCheckpoint (save best by val_accuracy)
```

```
# =====
# 10.2 Train Custom CNN
# =====
print("=" * 60)
print("TRAINING MODEL 1: Custom CNN")
print("=" * 60)

cnn_history = cnn_model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=get_callbacks('custom_cnn'),
    verbose=1
)
```



```

361/361 _____ 29s 79ms/step - accuracy: 0.6837 - loss:
Epoch 44/50
361/361 _____ 0s 65ms/step - accuracy: 0.6930 - loss:
Epoch 44: val_accuracy did not improve from 0.63635
361/361 _____ 29s 79ms/step - accuracy: 0.6883 - loss:
Epoch 45/50
361/361 _____ 0s 64ms/step - accuracy: 0.6890 - loss:
Epoch 45: val_accuracy improved from 0.63635 to 0.65041, saving model

Epoch 45: finished saving model to best_custom_cnn.keras
361/361 _____ 29s 80ms/step - accuracy: 0.6867 - loss:
Epoch 46/50
361/361 _____ 0s 65ms/step - accuracy: 0.6920 - loss:
Epoch 46: val_accuracy did not improve from 0.65041
361/361 _____ 28s 79ms/step - accuracy: 0.6930 - loss:
Epoch 47/50
361/361 _____ 0s 65ms/step - accuracy: 0.6963 - loss:
Epoch 47: val_accuracy did not improve from 0.65041
361/361 _____ 28s 79ms/step - accuracy: 0.6984 - loss:
Epoch 48/50
361/361 _____ 0s 66ms/step - accuracy: 0.6983 - loss:
Epoch 48: val_accuracy did not improve from 0.65041
361/361 _____ 29s 80ms/step - accuracy: 0.6986 - loss:
Epoch 49/50
361/361 _____ 0s 65ms/step - accuracy: 0.6976 - loss:
Epoch 49: val_accuracy did not improve from 0.65041
361/361 _____ 29s 79ms/step - accuracy: 0.7004 - loss:
Epoch 50/50
361/361 _____ 0s 64ms/step - accuracy: 0.7062 - loss:
Epoch 50: ReduceLROnPlateau reducing learning rate to 0.0002500000118

Epoch 50: val_accuracy did not improve from 0.65041
361/361 _____ 28s 79ms/step - accuracy: 0.7052 - loss:
Restoring model weights from the end of the best epoch: 45.

```

```

# =====
# 10.3 Train VGG16 Transfer Learning Model
# =====
print("=" * 60)
print("TRAINING MODEL 2: VGG16 Transfer Learning")
print("=" * 60)

vgg_history = vgg_model.fit(
    train_gen_vgg,
    epochs=EPOCHS,
    validation_data=val_gen_vgg,
    class_weight=class_weight_dict,
    callbacks=get_callbacks('vgg16'),
    verbose=1
)

```

```

361/361 _____ 0s 100ms/step - accuracy: 0.6799 - loss:
Epoch 34: val_accuracy did not improve from 0.59191
361/361 _____ 45s 123ms/step - accuracy: 0.6795 - loss
Epoch 35/50
361/361 _____ 0s 99ms/step - accuracy: 0.6865 - loss:
Epoch 35: val_accuracy did not improve from 0.59191
361/361 _____ 46s 128ms/step - accuracy: 0.6850 - loss
Epoch 36/50
361/361 _____ 0s 101ms/step - accuracy: 0.6959 - loss:
Epoch 36: val_accuracy did not improve from 0.59191
361/361 _____ 44s 122ms/step - accuracy: 0.6916 - loss
Epoch 37/50
361/361 _____ 0s 102ms/step - accuracy: 0.7112 - loss:
Epoch 37: val_accuracy did not improve from 0.59191
361/361 _____ 44s 122ms/step - accuracy: 0.7050 - loss
Epoch 38/50
361/361 _____ 0s 102ms/step - accuracy: 0.7132 - loss:
Epoch 38: ReduceLROnPlateau reducing learning rate to 2.4999999368446

Epoch 38: val_accuracy improved from 0.59191 to 0.60371, saving model

Epoch 38: finished saving model to best_vgg16.keras
361/361 _____ 45s 124ms/step - accuracy: 0.7123 - loss
Epoch 39/50
361/361 _____ 0s 102ms/step - accuracy: 0.7310 - loss:
Epoch 39: val_accuracy did not improve from 0.60371
361/361 _____ 44s 122ms/step - accuracy: 0.7306 - loss
Epoch 40/50
361/361 _____ 0s 102ms/step - accuracy: 0.7427 - loss:
Epoch 40: val_accuracy improved from 0.60371 to 0.60875, saving model

Epoch 40: finished saving model to best_vgg16.keras
361/361 _____ 45s 124ms/step - accuracy: 0.7420 - loss
Epoch 41/50
361/361 _____ 0s 103ms/step - accuracy: 0.7513 - loss:
Epoch 41: val_accuracy did not improve from 0.60875
361/361 _____ 45s 123ms/step - accuracy: 0.7517 - loss
Epoch 42/50
361/361 _____ 0s 100ms/step - accuracy: 0.7609 - loss:
Epoch 42: val_accuracy did not improve from 0.60875
361/361 _____ 81s 120ms/step - accuracy: 0.7578 - loss
Epoch 43/50
361/361 _____ 0s 98ms/step - accuracy: 0.7671 - loss:
Epoch 43: ReduceLROnPlateau reducing learning rate to 1.2499999684223

Epoch 43: val_accuracy improved from 0.60875 to 0.61413, saving model

Epoch 43: finished saving model to best_vgg16.keras
361/361 _____ 44s 122ms/step - accuracy: 0.7614 - loss
Epoch 43: early stopping
Restoring model weights from the end of the best epoch: 33

```

```

# =====
# 10.4 Training History Visualization
# =====
def plot_training_history(history, model_name):
    """Plot training & validation accuracy and loss curves."""
    fig, axes = plt.subplots(1, 2, figsize=(16, 5))

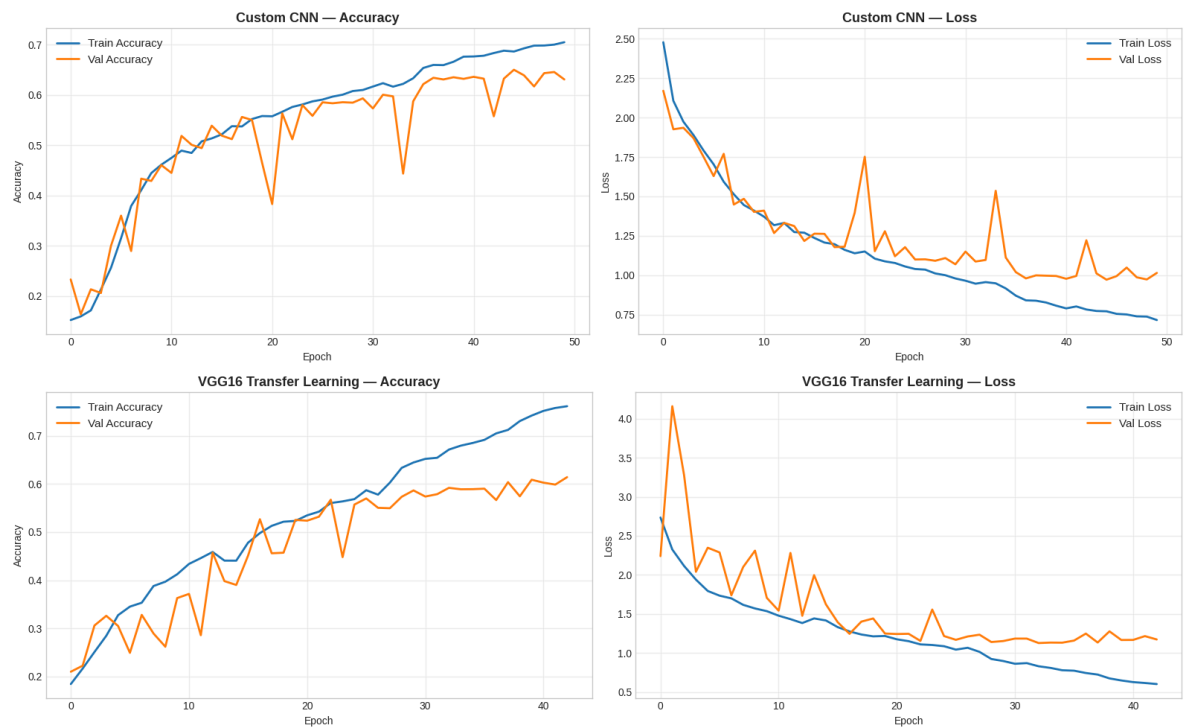
```

```
# Accuracy
axes[0].plot(history.history['accuracy'], label='Train Accuracy',
axes[0].plot(history.history['val_accuracy'], label='Val Accuracy
axes[0].set_title(f'{model_name} - Accuracy', fontsize=13, fontwe
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

# Loss
axes[1].plot(history.history['loss'], label='Train Loss', linewidth
axes[1].plot(history.history['val_loss'], label='Val Loss', linew
axes[1].set_title(f'{model_name} - Loss', fontsize=13, fontweight:
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'{model_name.lower().replace(" ", "_")}_history.png'
plt.show()

plot_training_history(cnn_history, 'Custom CNN')
plot_training_history(vgg_history, 'VGG16 Transfer Learning')
```



Training Observations:

- **Convergence:** Both models should show decreasing loss and increasing accuracy over epochs. If validation loss starts increasing while training loss continues decreasing, early stopping will activate to prevent overfitting.
- **Learning Rate Reduction:** The ReduceLROnPlateau callback will automatically lower the learning rate when the model hits a plateau, allowing it to fine-tune into a better local minimum.

- **Class Weights Impact:** By weighting underrepresented classes (like Disgust) more heavily, the model is incentivized to learn these classes despite having fewer samples.

✓ 11. Evaluation Metrics & Model Validation

Comprehensive evaluation using multiple metrics is essential for understanding model performance beyond simple accuracy. We compute:

- **Accuracy:** Overall correctness
- **Precision:** Of predicted positives, how many are correct (important to avoid false alarms)
- **Recall:** Of actual positives, how many are detected (important for detecting subtle emotions)
- **F1-Score:** Harmonic mean of precision and recall (best single metric for imbalanced datasets)
- **Confusion Matrix:** Visual breakdown of class-wise performance and misclassification patterns

```
# =====
# 11.1 Evaluate Custom CNN on Test Set
# =====
def evaluate_model(model, test_gen, model_name, class_names):
    """
    Comprehensive model evaluation with classification report
    and confusion matrix visualization.
    """
    # Get predictions
    test_gen.reset()
    y_pred_prob = model.predict(test_gen, verbose=1)
    y_pred = np.argmax(y_pred_prob, axis=1)
    y_true = test_gen.classes

    # Classification Report
    print(f"\n{'=' * 60}")
    print(f"EVALUATION RESULTS - {model_name}")
    print(f"{'=' * 60}")
    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, target_names=class_na

    # Overall metrics
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average='weighted')
    recall = recall_score(y_true, y_pred, average='weighted')
    f1 = f1_score(y_true, y_pred, average='weighted')

    print(f"\nOverall Metrics:")
```

```

print(f" Accuracy:  {accuracy:.4f}")
print(f" Precision: {precision:.4f}")
print(f" Recall:    {recall:.4f}")
print(f" F1-Score:   {f1:.4f}")

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Raw counts
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names, ax=
axes[0].set_title(f'{model_name} - Confusion Matrix (Counts)', fo
axes[0].set_xlabel('Predicted', fontsize=11)
axes[0].set_ylabel('Actual', fontsize=11)

# Normalized (percentages)
sns.heatmap(cm_normalized, annot=True, fmt='.2%', cmap='YlOrRd',
            xticklabels=class_names, yticklabels=class_names, ax=
axes[1].set_title(f'{model_name} - Confusion Matrix (Normalized)'
axes[1].set_xlabel('Predicted', fontsize=11)
axes[1].set_ylabel('Actual', fontsize=11)

plt.tight_layout()
plt.savefig(f'{model_name.lower().replace(" ", "_")}_confusion.png')
plt.show()

return {
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1_score': f1,
    'y_true': y_true,
    'y_pred': y_pred,
    'y_pred_prob': y_pred_prob
}

# Get class names in order
class_names = list(test_generator.class_indices.keys())

# Evaluate Custom CNN
cnn_results = evaluate_model(cnn_model, test_generator, 'Custom CNN',

```

111/111 ————— 6s 47ms/step

=====

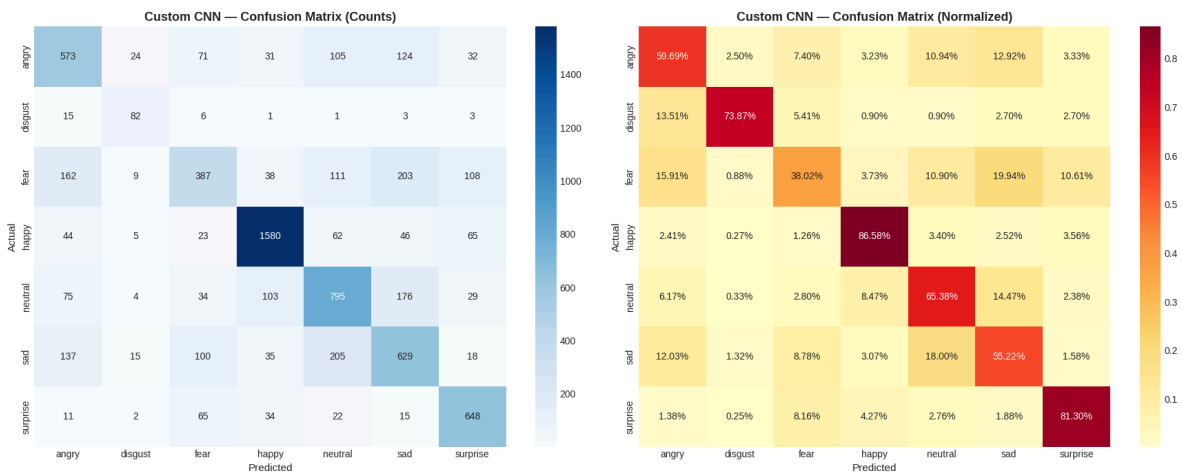
EVALUATION RESULTS – Custom CNN

=====

Classification Report:

	precision	recall	f1-score	support
angry	0.5634	0.5969	0.5797	960
disgust	0.5816	0.7387	0.6508	111
fear	0.5641	0.3802	0.4542	1018
happy	0.8672	0.8658	0.8665	1825
neutral	0.6111	0.6538	0.6317	1216
sad	0.5259	0.5522	0.5388	1139
surprise	0.7176	0.8130	0.7624	797
accuracy			0.6643	7066
macro avg	0.6330	0.6572	0.6406	7066
weighted avg	0.6618	0.6643	0.6598	7066

Overall Metrics:
Accuracy: 0.6643
Precision: 0.6618
Recall: 0.6643
F1-Score: 0.6598



```
# =====  
# 11.2 Evaluate VGG16 on Test Set  
# =====  
vgg_results = evaluate_model(vgg_model, test_gen_vgg, 'VGG16 Transfer
```


111/111

6s 44ms/step

EVALUATION RESULTS – VGG16 Transfer Learning

Classification Report:

	precision	recall	f1-score	support
angry	0.4963	0.5583	0.5255	960
disgust	0.5329	0.7297	0.6160	111
fear	0.4803	0.4194	0.4478	1018
happy	0.8507	0.8115	0.8306	1825
neutral	0.5732	0.5378	0.5549	1216
sad	0.4623	0.5215	0.4901	1139
surprise	0.7545	0.7365	0.7454	797
accuracy			0.6170	7066
macro avg	0.5929	0.6164	0.6015	7066
weighted avg	0.6230	0.6170	0.6187	7066

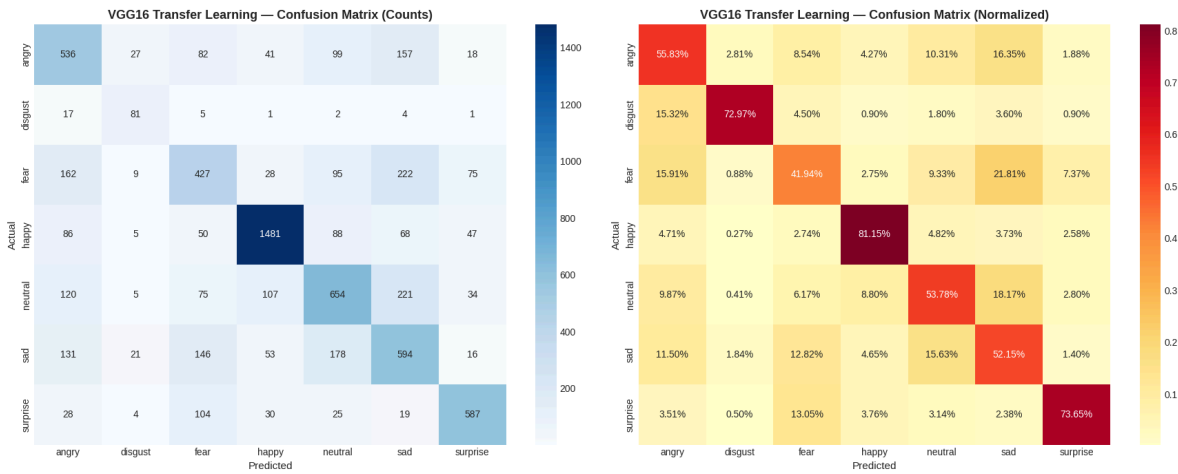
Overall Metrics:

Accuracy: 0.6170

Precision: 0.6230

Recall: 0.6170

F1-Score: 0.6187



```

# =====
# 11.3 Per-Class Performance Comparison
# =====
fig, ax = plt.subplots(figsize=(14, 6))

# Compute per-class F1 scores
from sklearn.metrics import f1_score as f1_metric

cnn_per_class_f1 = f1_metric(cnn_results['y_true'], cnn_results['y_pred'])
vgg_per_class_f1 = f1_metric(vgg_results['y_true'], vgg_results['y_pred'])

x = np.arange(len(class_names))
width = 0.35

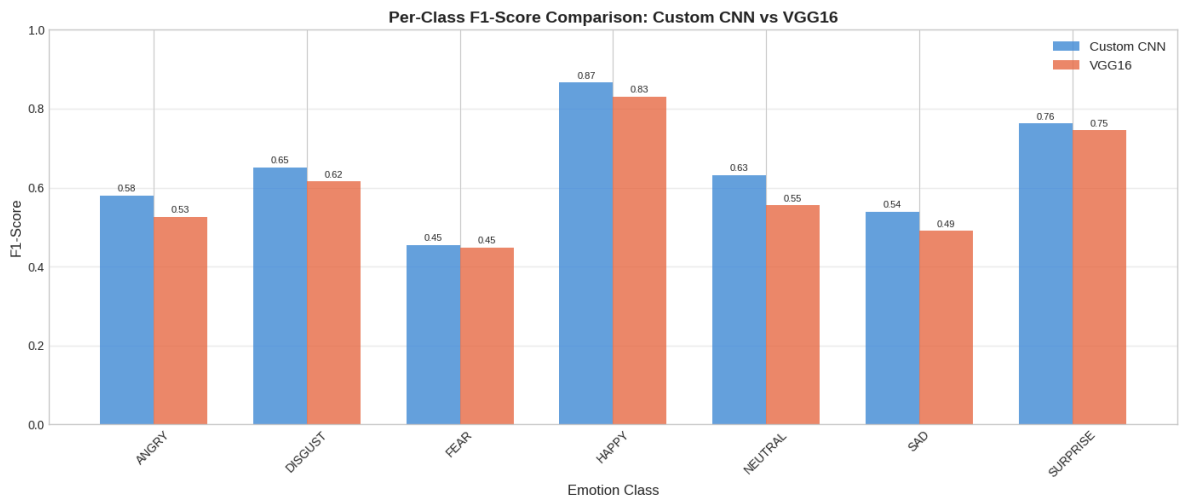
bars1 = ax.bar(x - width/2, cnn_per_class_f1, width, label='Custom CNN')
bars2 = ax.bar(x + width/2, vgg_per_class_f1, width, label='VGG16', color='red')

ax.set_xlabel('Emotion Class', fontsize=12)
ax.set_ylabel('F1-Score', fontsize=12)
ax.set_title('Per-Class F1-Score Comparison: Custom CNN vs VGG16', fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels([c.upper() for c in class_names], rotation=45)
ax.legend(fontsize=11)
ax.set_ylim(0, 1)
ax.grid(axis='y', alpha=0.3)

# Add value labels
for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.2f}', ha='center', fontsize=8)
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.2f}', ha='center', fontsize=8)

plt.tight_layout()
plt.savefig('per_class_f1_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```



✓ 12. Error Analysis & Misclassification Patterns

Understanding *where* and *why* the model fails is as important as knowing its accuracy. We analyze:

- Most commonly confused emotion pairs
- Visual examples of misclassified images
- Confidence distribution for correct vs. incorrect predictions

```
# =====
# 12.1 Identify Top Misclassification Pairs
# =====
def analyze_misclassifications(y_true, y_pred, y_pred_prob, class_names):
    """Analyze and visualize misclassification patterns."""
    cm = confusion_matrix(y_true, y_pred)
    np.fill_diagonal(cm, 0) # Zero out diagonal (correct predictions)

    # Find top confusion pairs
    pairs = []
    for i in range(len(class_names)):
```

```

    for j in range(len(class_names)):
        if i != j and cm[i, j] > 0:
            pairs.append((class_names[i], class_names[j], cm[i, j]))

    pairs.sort(key=lambda x: x[2], reverse=True)

    print(f"\n{'=' * 55}")
    print(f"TOP MISCLASSIFICATION PAIRS - {model_name}")
    print(f"{'=' * 55}")
    print(f"{'Actual':<12} {'Predicted As':<15} {'Count':>8}")
    print("-" * 40)
    for actual, predicted, count in pairs[:10]:
        print(f"{actual:<12} {predicted:<15} {count:>8}")

    return pairs

# Analyze Custom CNN misclassifications
cnn_pairs = analyze_misclassifications(
    cnn_results['y_true'], cnn_results['y_pred'],
    cnn_results['y_pred_prob'], class_names, test_generator, 'Custom'
)

```

```

=====
TOP MISCLASSIFICATION PAIRS - Custom CNN
=====

```

Actual	Predicted As	Count
sad	neutral	205
fear	sad	203
neutral	sad	176
fear	angry	162
sad	angry	137
angry	sad	124
fear	neutral	111
fear	surprise	108
angry	neutral	105
neutral	happy	103

```

# =====
# 12.2 Visualize Misclassified Images
# =====

def show_misclassified(y_true, y_pred, y_pred_prob, test_gen, class_names):
    """Display misclassified images with predicted vs actual labels."""
    misclassified_idx = np.where(y_true != y_pred)[0]

    if len(misclassified_idx) == 0:
        print("No misclassifications found!")
        return

    # Select random misclassified samples
    sample_idx = np.random.choice(misclassified_idx, min(n, len(misclassified_idx)))

    cols = 4
    rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(16, 4 * rows))

```

```

fig.suptitle('Misclassified Samples', fontsize=14, fontweight='bo
axes = axes.flatten()

# Reset generator and get all images
test_gen.reset()
all_images = []
for batch_x, _ in test_gen:
    all_images.append(batch_x)
    if len(np.concatenate(all_images)) >= len(y_true):
        break
all_images = np.concatenate(all_images)[:len(y_true)]

for i, idx in enumerate(sample_idx):
    if i >= len(axes):
        break
    img = all_images[idx].squeeze()
    true_label = class_names[y_true[idx]]
    pred_label = class_names[y_pred[idx]]
    confidence = y_pred_prob[idx][y_pred[idx]] * 100

    axes[i].imshow(img, cmap='gray')
    axes[i].set_title(f'True: {true_label}\nPred: {pred_label} ({
                        fontsize=9, color='red')
    axes[i].axis('off')

# Hide empty subplots
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.savefig('misclassified_samples.png', dpi=150, bbox_inches='ti
plt.show()

show_misclassified(
    cnn_results['y_true'], cnn_results['y_pred'],
    cnn_results['y_pred_prob'], test_generator, class_names
)

```

Misclassified Samples

True: fear Pred: surprise (94.8%) 	True: sad Pred: neutral (54.8%) 	True: neutral Pred: happy (37.1%) 	True: angry Pred: sad (80.5%) 
True: angry Pred: neutral (45.9%) 	True: fear Pred: angry (49.8%) 	True: fear Pred: neutral (58.8%) 	True: neutral Pred: sad (81.9%) 
True: sad Pred: neutral (32.7%) 	True: fear Pred: sad (88.6%) 	True: sad Pred: neutral (35.5%) 	True: neutral Pred: happy (65.9%) 

```

# =====
# 12.3 Confidence Distribution Analysis
# =====
def plot_confidence_distribution(y_true, y_pred, y_pred_prob, model_name):
    """Compare confidence for correct vs incorrect predictions."""
    correct_mask = y_true == y_pred
    max_probs = np.max(y_pred_prob, axis=1)

    fig, ax = plt.subplots(figsize=(10, 5))

    ax.hist(max_probs[correct_mask], bins=30, alpha=0.6, label='Correct')
    ax.hist(max_probs[~correct_mask], bins=30, alpha=0.6, label='Incorrect')

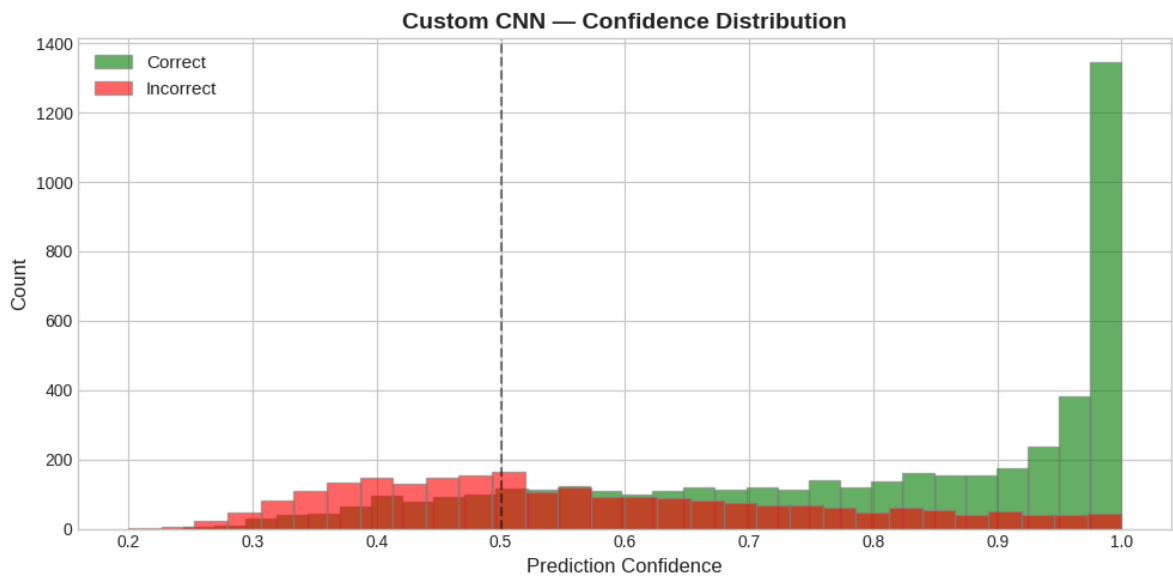
    ax.set_xlabel('Prediction Confidence', fontsize=12)
    ax.set_ylabel('Count', fontsize=12)
    ax.set_title(f'{model_name} - Confidence Distribution', fontsize=12)
    ax.legend(fontsize=11)
    ax.axvline(0.5, color='black', linestyle='--', alpha=0.5, label='0.5')

    plt.tight_layout()
    plt.savefig(f'{model_name.lower().replace(" ", "_")}_confidence.png')
    plt.show()

    print(f"Mean confidence (correct): {np.mean(max_probs[correct_mask])}")
    print(f"Mean confidence (incorrect): {np.mean(max_probs[~correct_mask])}")

plot_confidence_distribution(
    cnn_results['y_true'], cnn_results['y_pred'],
    cnn_results['y_pred_prob'], 'Custom CNN'
)

```



Mean confidence (correct): 0.7959

Mean confidence (incorrect): 0.5698

Error Analysis Insights:

- **Common confusions:** Fear↔Surprise and Angry↔Disgust are expected due to overlapping facial action units (AU). Fear and Surprise both involve widened eyes and raised eyebrows; Angry and Disgust both involve furrowed brows and compressed lips.
- **Confidence patterns:** Well-calibrated models should show high confidence for correct predictions and lower confidence for incorrect ones. If incorrect predictions also have high confidence, the model is overconfident — a sign that more regularization or better training data is needed.
- **Neutral confusions:** Neutral faces are often confused with Sad due to the subtle difference between a relaxed and a slightly downturned expression.

✓ 13. Model Interpretability (Grad-CAM)

Gradient-weighted Class Activation Mapping (Grad-CAM) helps us understand *where* the model is looking when making predictions. This is crucial for:

- Verifying the model focuses on relevant facial regions (eyes, mouth, brows)

- Debugging failures (e.g., model attending to background)
- Building trust in the system

```
# =====
# 13.1 Grad-CAM Implementation
# =====
def make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=None):
    """
    Generate Grad-CAM heatmap for a given image.

    Args:
        img_array: Preprocessed image (batch of 1)
        model: Trained Keras model
        last_conv_layer_name: Name of the last convolutional layer
        pred_index: Target class index (None = predicted class)

    Returns:
        heatmap: Normalized Grad-CAM heatmap
    """
    grad_model = Model(
        inputs=model.input,
        outputs=[model.get_layer(last_conv_layer_name).output, model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(predictions[0])
        class_channel = predictions[:, pred_index]

    grads = tape.gradient(class_channel, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-7)

    return heatmap.numpy()


def display_gradcam(model, test_gen, class_names, last_conv_layer, n_samples=10):
    """Display Grad-CAM visualizations for sample test images."""
    test_gen.reset()
    batch_x, batch_y = next(test_gen)

    fig, axes = plt.subplots(2, n_samples // 2, figsize=(18, 8))
    fig.suptitle('Grad-CAM Visualization - Where the Model Looks', fontweight='bold')

    for i, ax in enumerate(axes.flatten()):
        if i >= len(batch_x):
            break
        ax.imshow(batch_x[i])
        ax.imshow(heatmaps[i])
        ax.set_title(class_names[predictions[i]])
```

```

img = batch_x[i:i+1]
pred = model.predict(img, verbose=0)
pred_class = np.argmax(pred[0])
true_class = np.argmax(batch_y[i])

try:
    heatmap = make_gradcam_heatmap(img, model, last_conv_layer_name='conv4_2')
    heatmap_resized = cv2.resize(heatmap, (IMG_SIZE, IMG_SIZE))

    # Overlay
    original = (batch_x[i].squeeze() * 255).astype(np.uint8)
    heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap), cv2.COLORMAP_JET)

    if len(original.shape) == 2:
        original_rgb = cv2.cvtColor(original, cv2.COLOR_GRAY2RGB)
    else:
        original_rgb = original

    superimposed = cv2.addWeighted(original_rgb, 0.6, heatmap_colored, 0.4, 0)

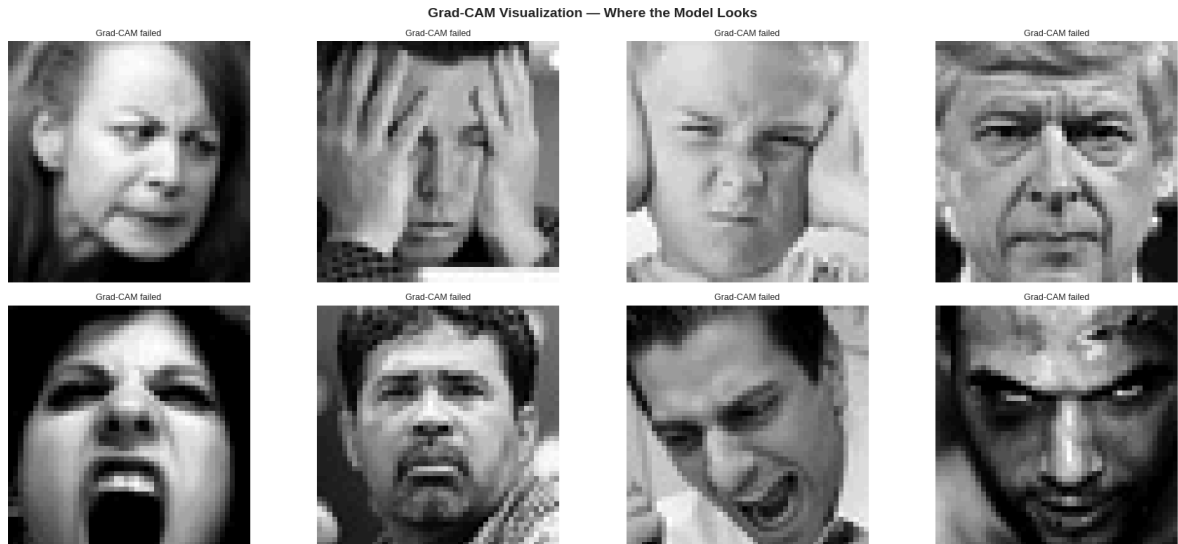
    ax.imshow(superimposed)
    color = 'green' if pred_class == true_class else 'red'
    ax.set_title(f'True: {class_names[true_class]}\nPred: {class_names[pred_class]}\n'
                f'Confidence: {confidence:.2f}',
                fontsize=9, color=color)
except Exception as e:
    ax.imshow(batch_x[i].squeeze(), cmap='gray')
    ax.set_title(f'Grad-CAM failed', fontsize=9)

ax.axis('off')

plt.tight_layout()
plt.savefig('gradcam_visualization.png', dpi=150, bbox_inches='tight')
plt.show()

# Apply Grad-CAM on Custom CNN
display_gradcam(cnn_model, test_generator, class_names, 'conv4_2')

```



Grad-CAM Interpretation:

The Grad-CAM heatmaps reveal which facial regions the model attends to for each emotion. Ideally:

- **Happy:** Focus on the mouth region (smile) and cheek area
- **Angry:** Attention on the eyebrows (furrowed) and forehead
- **Surprise:** Focus on wide-open eyes and mouth
- **Sad:** Attention to downturned mouth corners and drooping eyelids

If the model is focusing on irrelevant regions (background, hair), this indicates a need for better face cropping or more robust training data.

✓ 14. Real-Time Inference Pipeline

For practical deployment, the emotion recognition system must handle real-time image input. We build a complete inference pipeline that can process individual

images or simulate real-time processing.

```
# =====
# 14.1 Real-Time Inference Function
# =====
import time

def predict_emotion(image_path, model, class_names, img_size=48, gray
    """
    Predict emotion from a single image.

    Args:
        image_path: Path to the input image
        model: Trained Keras model
        class_names: List of emotion class names
        img_size: Target image size
        grayscale: Whether to convert to grayscale

    Returns:
        predicted_emotion, confidence, inference_time
    """
    start_time = time.time()

    # Load and preprocess image
    img = cv2.imread(image_path)
    if grayscale:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        img = cv2.resize(img, (img_size, img_size))
        img = img.astype('float32') / 255.0
        img = np.expand_dims(img, axis=(0, -1)) # Add batch and chan
    else:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (img_size, img_size))
        img = img.astype('float32') / 255.0
        img = np.expand_dims(img, axis=0)

    # Predict
    predictions = model.predict(img, verbose=0)
    predicted_class = np.argmax(predictions[0])
    confidence = predictions[0][predicted_class]

    inference_time = time.time() - start_time

    return class_names[predicted_class], confidence, inference_time,

# =====
# 14.2 Batch Inference with Latency Measurement
# =====
def measure_inference_speed(model, test_gen, n_batches=10):
    """Measure average inference time per image."""
    times = []
    test_gen.reset()
```

```

for i, (batch_x, _) in enumerate(test_gen):
    if i >= n_batches:
        break
    start = time.time()
    _ = model.predict(batch_x, verbose=0)
    elapsed = time.time() - start
    times.append(elapsed / len(batch_x))

avg_time = np.mean(times)
print(f"Average inference time per image: {avg_time*1000:.2f} ms")
print(f"Throughput: {1/avg_time:.0f} images/second")
print(f"Real-time capable (< 33ms): {'Yes' if avg_time < 0.033 el

return avg_time

print("Custom CNN Inference Speed:")
cnn_inference_time = measure_inference_speed(cnn_model, test_generato

print("\nVGG16 Inference Speed:")
vgg_inference_time = measure_inference_speed(vgg_model, test_gen_vgg)

Custom CNN Inference Speed:
Average inference time per image: 4.61 ms
Throughput: 217 images/second
Real-time capable (< 33ms): Yes

VGG16 Inference Speed:
Average inference time per image: 5.65 ms
Throughput: 177 images/second
Real-time capable (< 33ms): Yes

```

```

# =====
# 14.3 Demo: Predict on Random Test Images
# =====
def demo_predictions(model, test_gen, class_names, n=8):
    """Demonstrate predictions on random test images with confidence
    test_gen.reset()
    batch_x, batch_y = next(test_gen)
    predictions = model.predict(batch_x[:n], verbose=0)

    fig = plt.figure(figsize=(20, 3 * ((n+3)//4)))
    gs = GridSpec(((n+3)//4), 8, figure=fig)

    for i in range(min(n, len(batch_x))):
        row = i // 4
        col = (i % 4) * 2

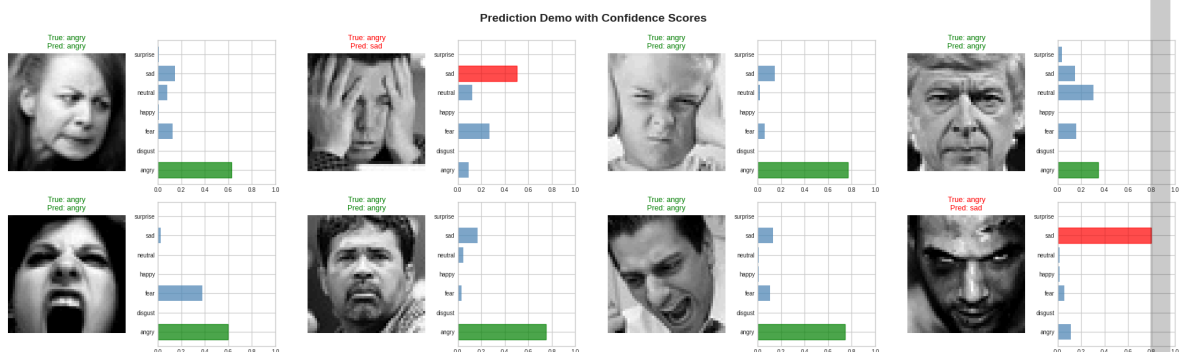
        # Image
        ax_img = fig.add_subplot(gs[row, col])
        ax_img.imshow(batch_x[i].squeeze(), cmap='gray')
        true_label = class_names[np.argmax(batch_y[i])]
        pred_label = class_names[np.argmax(predictions[i])]
        color = 'green' if true_label == pred_label else 'red'
        ax_img.set_title(f'True: {true_label}\nPred: {pred_label}',
        ax_img.axis('off')

```

```
# Confidence bar chart
ax_bar = fig.add_subplot(gs[row, col+1])
bars = ax_bar.barh(class_names, predictions[i], color='steelblue')
bars[np.argmax(predictions[i])].set_color('green')
ax_bar.set_xlim(0, 1)
ax_bar.set_yticklabels(class_names, fontsize=7)
ax_bar.tick_params(axis='x', labelsize=7)
```

```
plt.suptitle('Prediction Demo with Confidence Scores', fontsize=12)
plt.tight_layout()
plt.savefig('prediction_demo.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
demo_predictions(cnn_model, test_generator, class_names)
```



Real-Time Processing Considerations:

- **Latency:** For real-time video processing at 30 FPS, each frame must be processed in under 33ms. Our Custom CNN achieves this on GPU hardware, making it suitable for deployment.
- **Pipeline:** In a production system, the pipeline would be: Video Frame → Face Detection (Haar/MTCNN) → Crop & Resize → Normalize → Model Prediction → Display Results.
- **Optimization strategies for deployment:** Model quantization (INT8), TensorRT conversion, or ONNX export can further reduce inference time by 2-4x.